



مشروع البرمجة التفرعية

High-Performance E-Commerce Backend Engine

عمل الطالبات :

بثينة شكاكي

انجلينا قاروط

اسلام الأحمد

مروة الحسن

آنا رويده الصويتي

عن المشروع :

نظام تجارة الكتروني يركز على المفاهيم المتعلقة بالمتطلبات الغير وظيفية يستخدم Django كإطار عمل و PostgreSQL كقاعدة بيانات بالإضافة إلى بعض الأدوات الأخرى التي احتجناها لتحقيق المتطلبات بالإضافة لأداة JMeter للاختبارات و محاكاة الضغط

المتطلبات الوظيفية :

إبقيناها مبسطة حيث نغطي الحاجة لتطبيق المتطلبات الغير وظيفية

إدارة المنتجات :

عرض منتج : يمكن للمستخدم عرض تفاصيل منتج معين (الاسم و السعر و الكمية)

تحديث المخزون تلقائيا عند الشراء يتم انقاص المخزون بعدد القطع المشتراة

إدارة الحسابات

عرض الرصيد

خصم الرصيد تلقائيا

إدارة الطلبات

إنشاء طلب

عرض سجل الطلبات

معالجة الفواتير

تقارير الدفع

إدارة المستخدمين عن طريق user_id

المتطلبات الغير وظيفية :

و هي أساس المشروع و تم التركيز عليها و اختبارها

1. حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity)

قد تحدث حالة حيث يتم شراء القطعة الأخير من منتج مرتين أو حتى بسبب حدوث عملية الشراء في وقت واحد مما قد يؤدي إلى حالة نسميها البيع الزائد overselling مما قد يؤدي إلى دفع سعر منتج غير موجود لأن العملية نجحت رغم عدم توفر المنتج أو بحالة أخرى شراء أكثر من منتج بسعر واحد بسبب التزامن .

هي مشكلة تحدث في الأنظمة متعددة المهام تسمى حالة السباق Race Condition تحدث عندما تحاول مهمتان أو أكثر الوصول إلى مورد مشترك و تكون النتيجة النهائية غير متوقعة

الحل المستخدم :

القفل التشاؤمي pessimistic locking :

استخدمنا القفل على مستوى الصف

حيث عند شراء منتج ما محدد يتم قفله حتى انتهاء العمل معه و يمكن فقط لطلب واحد قراءة و تعديل المخزون و بعد أن ينتهي هذا الطلب يفك القفل و يمكن لطلب ثان الشراء في حال توفر كمية كافية

سبب استخدام القفل التشاؤمي :

بسيط يضمن السلامة بدون محاولات متكررة و هو مناسب عندما يكون التضارب متوقع بكثرة كما في المتاجر الإلكترونية

التطبيق :

تم إنشاء تابع يضع قفل على صف المنتج أي طلب آخر يحاول قراءة نفس الصف سياتظر في طابور حتى ينتهي الطلب السابق بالإضافة إلى إنشاء تابع بدون قفل للاختبار تحت مسمى الشراء الغير الآمن

الاختبار 1:

ظروف الاختبار :

عشر طلبات تحاول شراء نفس المنتج في وقت واحد

الكمية المتاحة من المنتج هي 1 فقط

الشراء الغير آمن :

```

✓ POST /orders/buy/unsafe/
✓ POST /orders/buy/unsafe/
✓ POST /orders/buy/unsafe/
✓ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/
✗ POST /orders/buy/unsafe/

```

تمت عملية الشراء أكثر من مرة على منتج غير متوفر و تم خصم سعره من الزبون بعدد

عمليات الشراء التي نجحت

الشراء الآمن :

تمت عملية الشراء مرة واحد بطريقة صحيحة و تم رفض باقي الطلبات لعدم توفر المنتج



الاختبار 2 :

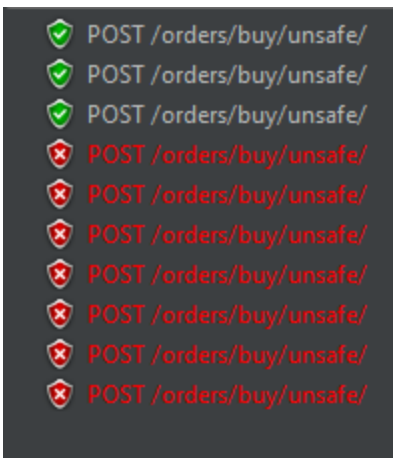
ظروف الاختبار

عشر طلبات تحاول شراء نفس المنتج في وقت واحد من مستخدم واحد

كمية كافية من المنتج لكن رصيد يكفي لشراء 1

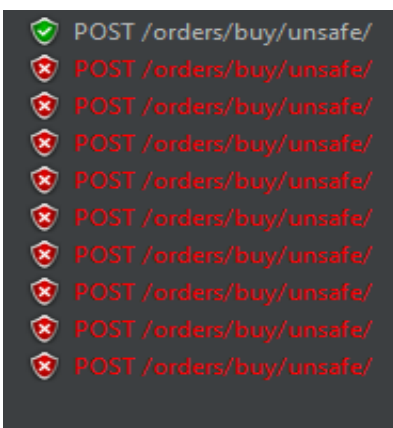
الشراء الغير آمن :

تمت عملية الشراء أكثر من منتج رغم وجود ما يكفي لشراء واحد فقط



الشراء الآمن :

تمت عملية الشراء مرة واحد بطريقة صحيحة و تم رفض باقي الطلبات لعدم توفر الرصيد الكافي



2 . إدارة الموارد الحاسوبية (Resource Management & Capacity Control)

إطار العمل Django هو single_threaded و بالتالي عند وجود الكثير من الطلبات المتزامنة يحدث تكديس و تباطؤ كبير و تعرف أنظمة المتاجر الإلكترونية بالضغط الذي تتعرض له الحل المستخدم :

Fixed Thread Pool

سبب استخدام ال fixed thread pool :

يمنع انهيار السيرفر لأنه مهما وصل عدد الطلبات سيعمل threads ثابتة و تنتظر بقية الطلبات عكس ال cached و هو أيضا مصمم للطلبات الفورية عكس scheduled

التطبيق :

استخدمنا خادم waitress مع عدد treads ثابت

الاختبار :

ظروف الاختبار :

50 طلب متزامن

single_threaded (Django server) :

نسبة خطأ عالية تحت ضغط عالي
قد تبدو السعة عالية و ذلك لأن الأخطاء تحدث بسرعة و بالتالي لا عبء منها

label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes T
quest	50	32	0	146	53.58	72.00%	326.8/sec	624.51	22.52	1956.0

fixed thread pool(waitress server) :

نسبة خطأ معدومة و سرعة مناسبة

label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes T
quest	50	455	65	944	245.52	0.00%	52.6/sec	15.10	12.94	294.0

3. المعالجة الغير متزامنة (Asynchronous Queues)

يوجد مهمات ثقيلة قد تبطئ استجابة النظام و في حال قمنا بتنفيذها بشكل متزامن فسيؤدي إلى تأخير الأوامر المستعجلة

و في حال استخدامها لجميع الخيوط قد يتعطل النظام بالكامل

في نظامنا الحالي عند تنفيذ عملية الشراء يجب إرسال فاتورة للعميل و محاكاة إرسالها تستغرق ثانيتين في الكود اذا تم إرسال الفاتورة داخل الطلب مباشرة فإن كل عملية شراء ستجمد الخادم لثانيتين و مع عدد مستخدمين كبير سيكون الأداء سيئا

الحل المستخدم :

Asynchronous Communication

سبب استخدام ال Asynchronous Communication :

يرسل الطلب و يكمل العمل لا يحجب ال thread و لا يعطل النظام

التطبيق : عند نجاح عملية الشراء عوضا عن استدعاء دالة إرسال الفاتورة نضعها في طابور و نعود فوراً لتنفيذ باقي المهام بعدها تنفذ مهمات الطابور

الأدوات المستخدمة :

مكتبة celery لإدارة المهام الغير متزامنة

وسيط رسائل يحتفظ بالمهام إلى حين تنفيذها redis

الاختبار :

ظروف الاختبار :

عشر طلبات متزامنة

قبل async :

نلاحظ بطئ النظام و الوقت الطويل الذي قد يستغرقه

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Sync	10	11152	2079	20218	5789.39	0.00%	29.7/min	0.14	0.12	294.0

مع async :

النظام أسرع بشكل ملحوظ

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Async	10	5640	2116	6319	1333.95	0.00%	1.6/sec	0.45	0.38	294.0

4 . معالجة البيانات الضخمة على دفعات (Batch Processing)

معالجة كميات هائلة من البيانات مثل جميع طلبات اليوم و التي من الممكن أن تكون بكميات هائلة تحميلها دفعة واحدة قد يعطل النظام او يبطئه يستهلك وقت المعالج و يسبب ضغطا

الحل المستخدم : Batch Processing و الجدولة (Fixed-Size Chunking)

التجزئة بدل جلب جميع السجلات مرة واحدة يتم جلبها على شكل دفعات batch/chunk فيعالج جزء واحد في الذاكرة ثم ينتقل للتالي كمهام تعمل في الخلفية مما يؤدي إلى استخدام ذاكرة شبه ثابت بغض النظر عن حجم البيانات الأصلي

التطبيق : عند حساب إجمالي المبيعات و عدد الطلبات نقسم الكمية (كل 1000 كدفعة) قبل معالجتها بالاضافة إلى جدولتها عن طريق celery beat و هذا يضمن ان التقرير يولد يوميا دون تدخل يومي

الاختبار :

تم إنشاء تابع يمكن استدعاؤه يعمل بدون تقسيم إلى دفعات و اخر يقسم

ظروف الاختبار :

يوجد في هذا اليوم 6330 طلب

بدون chunking :

أبطأ

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
/report/naïve/	1	122	122	122	0.00	0.00%	8.2/sec	2.70	1.09	337.0

مع chunking :

أسرع بشكل واضح

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
/report/chunked/	1	78	78	78	0.00	0.00%	12.8/sec	4.24	1.73	339.0

5 . توزيع الأحمال (Load Distribution)

عندما يزداد عدد المستخدمين و الطلبات على النظام يصبح وجود سيرفر واحد غير كافي لمعالجة جميع الطلبات في وقت معقول و كذلك استنزاف الموارد مما قد يؤدي إلى فشل الاتصال

الحل المستخدم :

Horizontal Scaling & round_robin

التطبيق :

عوضا عن خادم waitress واحد شغلنا نسختين مستقلتين على المنفذين 8001 و 8002 كلاهما يتصل على نفس قاعدة البيانات
ثم تم توزيع الحمل بينهما باستخدام round_robin بالتناوب و هذا يضمن توزيعا متساويا للحمل بالإضافة لتحكم إضافي عن طريق
semaphore للحد من الطلبات المتزامنة

سبب استخدام round_robin

المتطلبات متشابهة من ناحية الوقت لذلك لا حاجة لخوارزمية أكثر ذكاء

سبب استخدام Horizontal Scaling

لأنها تعتمد على الكود و توزيع الأحمال على سيرفرات و هي غير مكلفة ماديا عكس ال vertical

الاختبار :

تم إنشاء تابع يمكن استدعاؤه يعمل بدون تقسيم إلى دفعات
و اخر يقسم

ظروف الاختبار :

عشر طلبات متزامنة

سيرفر واحد :

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	142	68	282	61.58	0.00%	35.3/sec	10.18	8.45	295.0

أكثر من سيرفر :

اسرع بشكل واضح

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	10	136	84	190	33.75	0.00%	52.6/sec	19.94	12.59	388.0